Protocolo y Arquitectura Web

Modelo cliente-servidor, HTTP y HTTPS

ORGANIZACIÓN DEL CONTENIDO

Temario de la Clase

BLOQUE 1 Modelo cliente-servidor y ciclo completo de una petición →

BLOQUE 2 Anatomía de los mensajes HTTP y payload →

BLOQUE 3 Métodos, seguridad e idempotencia →

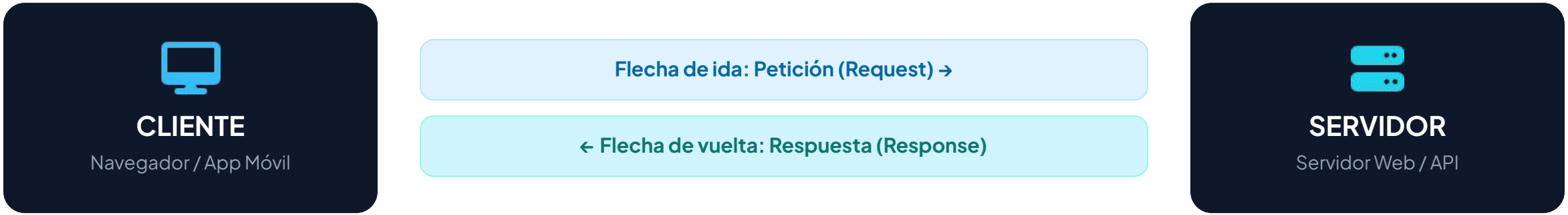
BLOQUE 4 Cabeceras y códigos de estado →

BLOQUE 5 HTTPS y cookies →

BLOQUE 1 | ARQUITECTURA BASE

El Modelo Cliente-Servidor (Esquema)

Es la arquitectura sobre la que se apoya toda la Web: dos actores con responsabilidades distintas que dialogan.



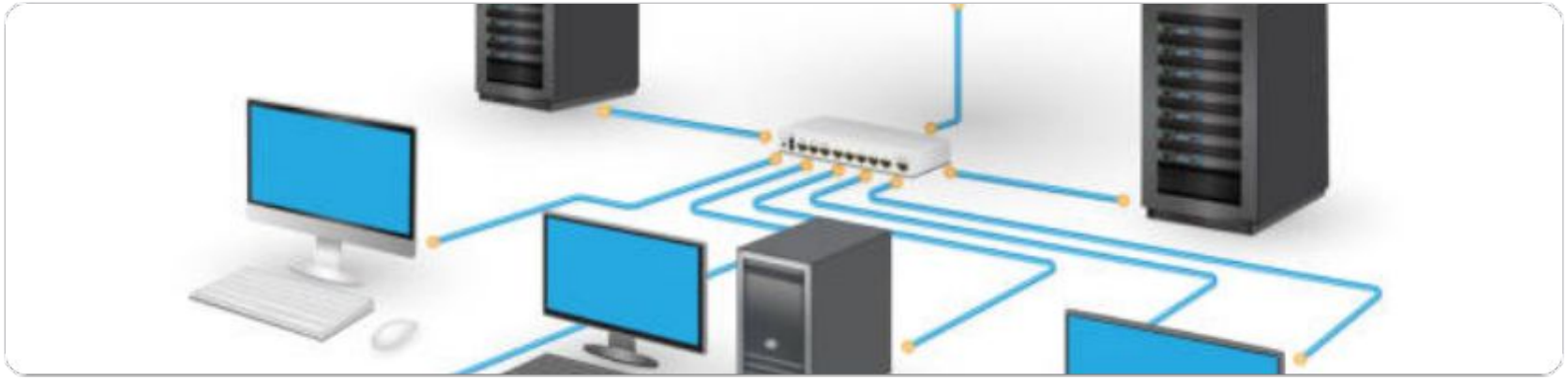
- **Iniciador:** La comunicación siempre empieza con una petición del cliente.
- **Indivisible:** Siempre hay un par inseparable: una petición y su respuesta.

BLOQUE 1 | ROL 1

El Cliente: Quien Solicita

Definición y Funciones

- **Qué es:** Navegador web (Chrome, Firefox), app móvil o aplicación de escritorio.
- **Función clave:** Construir la petición, enviarla por la red y renderizar la respuesta.
- **Inicia siempre:** En HTTP tradicional el servidor nunca envía datos si el cliente no los pidió explícitamente.



EJEMPLO

Cuando abrís Chrome y escribís una dirección, Chrome es el cliente; vos sos el usuario que opera la aplicación.

BLOQUE 1 | ROL 2

El Servidor: Quien Responde

Es una computadora (o grupo de computadoras) escuchando activamente peticiones en la red.

Responsabilidades

- **Procesamiento:** Aplica lógica de negocio y consulta bases de datos.
- **Resultado:** Genera una respuesta estructurada (JSON, HTML, archivos).
- **Centralización:** Garantiza la seguridad, validación de datos y persistencia.
- **Concurrencia:** Atiende a miles de clientes al mismo tiempo.

Infraestructura en Escucha

Permanece encendido en puertos estándar (80 para HTTP, 443 para HTTPS) esperando solicitudes.

EJEMPLO EN TIENDA ONLINE

El servidor valida tu pedido, controla el stock en su base de datos y confirma la compra emitiendo el comprobante.

BLOQUE 1 | CONCEPTOS CLAVE

Analogía del Restaurante

Una comparación cotidiana para recordar fácilmente los roles de la arquitectura web.



Comensal

Cliente: hace el pedido siguiendo el menú disponible.



Cocina

Servidor: prepara el plato solicitado y lo entrega.

Mozo

La Red: lleva y trae los mensajes, pero no cocina.



Menú y Reglas

Protocolo HTTP: reglas estandarizadas del salón.

⚠ CASOS DE ERROR EN EL RESTAURANTE

Pedís un plato que no existe en la carta: **"No lo tenemos" (404)**. La cocina está desbordada y no da abasto: **"No podemos atender" (503)**.

BLOQUE 1 | FLUJO DE DATOS

Ciclo Completo de una Petición Web (Esquema)

Antes de ver la página en pantalla pasan cinco etapas ordenadas secuencialmente.

1

DNS

Traduce dominio a IP (192.0.2.1).

2

TCP / TLS

Handshake y canal seguro.

3

HTTP Request

GET /index.html al servidor.

4

HTTP Response

200 OK + Payload HTML.

5

Render

El navegador dibuja la interfaz.

RESPUESTA AL PRESIONAR ENTER

Esta secuencia de 5 pasos constituye la respuesta técnica exacta a la pregunta clásica: *¿Qué sucede cuando escribís una URL y presionás Enter?*

BLOQUE 1 | CASO PRÁCTICO

Escenario Real del Ciclo Web

Seguimos un caso real: escribís https://mi-tienda.com/productos en el navegador.

- **Qué se pide:** La lista completa de productos del catálogo.
- **Protocolo utilizado:** HTTPS (HTTP sobre un canal seguro con cifrado).
- **Tiempo de ejecución:** Las 5 etapas transcurren en fracciones de segundo.

BLOQUE 1 | ETAPA 1

Etapa 1: Resolución DNS

DNS (Domain Name System) funciona como la agenda de contactos global de internet.

Problema

La red no entiende nombres alfanuméricos como mi-tienda.com, solo procesa direcciones IP numéricas.

Pregunta

El navegador consulta al servidor DNS resolver: *"¿A qué IP numérica corresponde el dominio mi-tienda.com?"*

Respuesta

El DNS retorna la IP pública exacta del servidor de destino, por ejemplo **190.102.45.12**.

ANALOGÍA DE LA AGENDA

Es exactamente igual a buscar a un amigo en tu agenda telefónica: tenés el nombre grabado, pero necesitás recuperar el número para llamarlo.

BLOQUE 1 | ETAPA 2

Etapa 2: Conexión TCP y TLS

Se construye la "tubería" cifrada de comunicación antes de transmitir ningún dato.

TCP (Transmission Control Protocol)

Establece un canal de transmisión ordenado y confiable mediante el protocolo Handshake de 3 vías.

TLS (Transport Layer Security)

Cifra la información enviada. Es la capa de seguridad que convierte a HTTP en HTTPS.

DIÁLOGO DE NEGOCIACIÓN (HANDSHAKE)

Cliente: *"¿Listo para hablar de forma segura?"* · Servidor: *"Sí, aquí están mis claves de cifrado y mi certificado digital"* · Cliente: *"Conexión segura establecida"*.

BLOQUE 1 | ETAPAS 3 Y 4

Etapas 3 y 4: Petición y Respuesta HTTP

Con el canal cifrado listo, ocurre el intercambio principal de mensajes.

Etapa 3: Petición

El cliente compone un mensaje formal estructurado solicitando el recurso específico.

Procesamiento

El servidor valida permisos, ejecuta la consulta SQL y arma el payload de resultado.

Etapa 4: Respuesta

Devuelve el código de estado (status code) junto con el contenido procesado.

MENSAJES INTERCAMBIADOS

Pedido: GET /productos HTTP/1.1 a la IP 190.102.45.12 · Respuesta: HTTP/1.1 200 OK + el HTML con el catálogo de productos.

BLOQUE 1 | ETAPA 5

Etapa 5: Renderizado en el Cliente

El trabajo de procesamiento se completa en el navegador web del cliente.

1. Descifra

Desencripta los paquetes que viajan por el canal TLS.

2. Interpreta

Lee HTML (estructura), CSS (estilos) y JavaScript.

3. Dibuja

Construye el árbol DOM y dibuja los píxeles en pantalla.

4. Pide Más

Inicia peticiones extra por cada imagen o script enlazado.

CARGA DE RECURSOS SECUNDARIOS

Una página compuesta por 1 HTML y 10 imágenes genera al menos 11 peticiones red independientes, no una sola solicitud.

BLOQUE 1 | ESTRUCTURA WEB

Anatomía de una URL

La URL (Uniform Resource Locator) identifica qué recurso se solicita y cómo acceder a él.

```
https://www.ejemplo.com:443/blog/post?id=7#comentarios
```

Esquema (https)
Protocolo de comunicación.

Host (www.ejemplo.com)
Nombre de dominio del servidor.

Puerto (:443)
Puerto de red (omitido si es estándar).

Ruta (/blog/post)
Ubicación interna del recurso.

Query (?id=7)
Parámetros clave=valor.

Fragmento (#coment)
Ancla interna (no viaja al servidor).

ACLARACIÓN DE PUERTOS

El puerto se omite en la barra del navegador cuando coincide con el estándar: puerto 80 para HTTP y puerto 443 para HTTPS.

BLOQUE 2 | MENSAJES HTTP

Anatomía de los Mensajes HTTP (Esquema)

Petición y respuesta comparten la misma estructura de tres partes fundamentales más una línea en blanco.

PETICIÓN (REQUEST)

```
METODO /RUTA HTTP/VERSION Clave: Valor Content-Type:
application/json (Línea en blanco obligatoria) [Cuerpo /
Payload]
```

RESPUESTA (RESPONSE)

```
HTTP/VERSION CODIGO MENSAJE Clave: Valor Content-Type:
application/json (Línea en blanco obligatoria) [Cuerpo /
Payload]
```

ANALOGÍA DE LA CARTA

Se lee exactamente como una carta postal: primero el sobre con la dirección y metadatos (primera línea y cabeceras), seguido por el contenido interno (cuerpo).

BLOQUE 2 | REQUEST

La Petición en Detalle (Request)

Texto en formato plano que envía el navegador para registrar un nuevo usuario.

```
POST /api/usuarios HTTP/1.1 Host: api.ejemplo.com
Content-Type: application/json Content-Length: 45 {
"nombre": "Carlos", "email": "carlos@mail.com" }
```

Método (POST): Acción deseada a realizar.

Ruta (/api/usuarios): Recurso en el servidor.

Versión: HTTP/1.1.

Cabeceras: Metadatos (Host, Content-Type).

Línea en blanco: Separador obligatorio.

Cuerpo: Objeto JSON con datos.

> 💬 **EXPLICACIÓN SEMÁNTICA**
> Se lee: *"Quiero crear (POST) un recurso en /api/usuarios hablando HTTP/1.1 y te adjunto estos datos en JSON"*.

BLOQUE 2 | RESPONSE

La Respuesta en Detalle (Response)

Texto plano que devuelve el servidor tras procesar exitosamente la creación del usuario.

```
HTTP/1.1 201 Created Content-Type: application/json;
charset=utf-8 Content-Length: 53 { "id": 101, "nombre":
"Carlos", "estado": "activo" }
```

Versión: Protocolo HTTP/1.1.

Código (201): Código numérico de estado.

Mensaje (Created): Razón legible del resultado.

Cabeceras: Formato y políticas de respuesta.

Cuerpo: Usuario generado con ID asignado.

> 💬 **EXPLICACIÓN SEMÁNTICA**
> El servidor responde: *"Operación exitosa, lo he creado (201); el nuevo usuario registrado posee el ID 101"*.

BLOQUE 2 | CONCEPTO CLAVE

¿Qué es el Payload?

Es la "carga útil" del mensaje: los datos sustanciales que realmente se quieren transmitir.

Definición Técnica

El cuerpo (body) de una petición o respuesta HTTP, excluyendo metadatos y cabeceras de red.

En Ambos Sentidos

Petición: Datos enviados por el cliente (formularios, JSON).

Respuesta: Recurso solicitado o confirmación procesada.

EJEMPLO DE FORMULARIO

Al completar un formulario de registro, tu nombre y contraseña son el payload del método POST. Representan el contenido real del paquete.

BLOQUE 3 | SEMÁNTICA REST

Métodos HTTP y Semántica REST

El método indica la acción deseada. Esta tabla resume sus propiedades fundamentales.

Método	Acción Principal	¿Payload en Petición?	¿Seguro?	¿Idempotente?
GET	Obtener / leer recursos	No recomendado	SÍ	SÍ
POST	Crear / procesar datos	Sí	NO	NO
PUT	Reemplazar recurso completo	Sí	NO	SÍ
PATCH	Modificar parcialmente	Sí	NO	NO
DELETE	Eliminar recurso	Opcional (raro)	NO	SÍ

BLOQUE 3 | SEGURIDAD EN MÉTODOS

¿Qué es un Método Seguro?

"Seguro" no significa "protegido": en la especificación HTTP significa que "no modifica datos".

Métodos Seguros (GET, HEAD)

Solo leen o consultan. Ejecutar un GET 100 veces consecutivas no altera en absoluto la base de datos.

Métodos No Seguros (POST, PUT, DELETE)

Alteran o modifican la información almacenada en los sistemas del servidor backend.

ANALOGÍA DEL MENÚ

Mirar la carta de un restaurante cien veces no cambia nada en la cocina (Seguro); hacer el pedido a la cocina sí altera los insumos (No Seguro).

BLOQUE 3 | IDEMPOTENCIA

Concepto de Idempotencia

Es la propiedad más abstracta; con ejemplos prácticos se vuelve totalmente concreta.

Idempotente (DELETE, PUT)

Borrar /usuarios/5 elimina al usuario. Repetir la orden 5 veces más deja al sistema exactamente en el mismo estado final: el usuario ya no existe.

No Idempotente (POST)

Enviar POST /pagos tres veces consecutivas por error en la red genera tres cobros distintos e independientes en el sistema.

ANALOGÍA DEL BOTÓN DEL ASCENSOR

Presionar el botón de llamada del ascensor una vez o presionarlo cinco veces seguidas produce exactamente el mismo resultado final. Es una acción idempotente.

BLOQUE 3 | MÉTODOS PRINCIPALES

GET y POST en Detalle

Representan los dos métodos con mayor frecuencia de uso en el navegador web.

GET (Leer)

Carga páginas o listas. Parámetros expuestos en la URL (/buscar?q=telefono).

Regla de Oro: Nunca envíes contraseñas en un GET; quedarían grabadas en la URL e historial.

POST (Crear)

Lleva el payload en el cuerpo del mensaje. No es seguro ni tampoco idempotente.

Seguridad: Lo que protege la privacidad de los datos es HTTPS, no el método HTTP usado.

USO EN NAVEGACIÓN

Navegar escribiendo una URL o haciendo clic en un enlace ejecuta un **GET**. Enviar un formulario de registro ejecuta un **POST**.

BLOQUE 3 | EDICIÓN DE DATOS

PUT y PATCH en Detalle

Ambos métodos modifican recursos existentes, pero con diferente alcance y semántica.

🔄PUT (Reemplazo Completo)

El payload debe contener la versión completa del objeto.

PUT es idempotente: enviar {id: 5, nombre: 'Juan'} diez veces mantiene el registro como 'Juan'.

🩹PATCH (Modificación Parcial)

Solo se transmiten los campos específicos que se desean alterar.

Suele enviar modificaciones incrementales como {"email": "nuevo@mail.com"}.

🪪 ANALOGÍA DE FICHA DE CONTACTO

Reemplazar toda la ficha de papel de un contacto por una totalmente nueva = **PUT**. Borrar solo su teléfono y escribir el nuevo = **PATCH**.

BLOQUE 3 | OTROS MÉTODOS

DELETE y Métodos Auxiliares

Completan el espectro de operaciones estandarizadas del protocolo HTTP.

DELETE

Elimina el recurso indicado en la URL. Rara vez lleva payload. Es de carácter idempotente.

HEAD

Igual a GET pero retorna únicamente las cabeceras, sin cuerpo de respuesta. Útil para verificaciones.

OPTIONS

Consulta los métodos permitidos por el servidor para un recurso

CASO PRÁCTICO DE HEAD

Comprobar si un archivo de 2 GB existe o si fue modificado sin necesidad de descargarlo completo: se emite un **HEAD**.

BLOQUE 4 | METADATOS

Cabeceras HTTP: Qué Son y Para Qué Sirven

Transportan metadatos esenciales sobre la petición, la respuesta, el cliente y el servidor.

Formato de Sintaxis

Pares clave-valor separados por dos puntos (Clave: Valor) enviados en líneas individuales.

Función en la Red

Definen reglas de negociación de formato, autenticación, gestión de caché y control de seguridad.

ANALOGÍA DE LA ETIQUETA POSTAL

Son las instrucciones pegadas en la encomienda: *"Frágil"*, *"Idioma: Español"*, *"Contenido: JSON"*. Indican cómo manipular el contenido.

BLOQUE 4 | REQUEST HEADERS

Cabeceras de Petición (Request Headers)

Información contextual enviada desde el cliente hacia el servidor web.

Host: api.ejemplo.com
Nombre del dominio de destino (obligatorio en HTTP/1.1).

User-Agent: Mozilla/5.0...
Identifica el navegador, motor y sistema operativo del cliente.

Accept: application/json
Informa qué tipo de formato espera recibir de respuesta.

Authorization: Bearer token...
Credenciales de seguridad para autenticar la solicitud.

ADAPTACIÓN DE IDIOMA
La cabecera Accept-Language: es-AR,es le indica automáticamente al servidor backend que entregue los textos traducidos al español.

BLOQUE 4 | RESPONSE HEADERS

Cabeceras de Respuesta (Response Headers)

Metadatos devueltos por el servidor para instruir al cliente cómo procesar la respuesta.

Content-Type: application/json
Especifica el formato exacto del payload adjunto.

Set-Cookie: session=xyz...
Ordena al navegador guardar una cookie de sesión.

Access-Control-Allow-Origin
Reglas de seguridad CORS para peticiones entre dominios.

Cache-Control: max-age=3600
Instrucciones sobre cuánto tiempo guardar en caché local.

OPTIMIZACIÓN CON CACHÉ
La cabecera Cache-Control: max-age=86400 le indica al navegador que guarde la imagen por 24 horas sin volver a pedirla al servidor.

BLOQUE 4 | STATUS CODES

Códigos de Estado HTTP: Cómo Leerlos

Número de 3 dígitos que indica el resultado de la petición. El primer dígito define la familia.

1×x	2×x	3×x	4×x	5×x
Informativo	Éxito	Redirección	Error Cliente	Error Servidor

REGLA NEMOTÉCNICA DE FAMILIAS

2×x: Todo salió bien · 3×x: El recurso se movió · 4×x: Te equivocaste vos (cliente) · 5×x: Se rompió el servidor backend.

BLOQUE 4 | ÉXITO Y REDIRECCIÓN

Códigos 2×x (Éxito) y 3×x (Redirección)

Indican resoluciones satisfactorias o reorientaciones de tráfico web.

Familia 2×x: Respuestas Exitosas

200 OK: Petición procesada correctamente.

201 Created: Recurso creado con éxito (POST/PUT).

204 No Content: Éxito sin cuerpo de respuesta.

Familia 3×x: Redirecciones

301 Moved Permanently: Cambio definitivo de URL.

302 Found: Redirección temporal.

304 Not Modified: El recurso en caché sigue vigente.

EJEMPLO DE 304

Al recargar una página, un estado **304 Not Modified** evita transferir archivos repetidos por la red, ahorrando ancho de banda y tiempo.

BLOQUE 4 | ERRORES DEL CLIENTE

Códigos 4×x: Errores del Cliente

Indican que la petición contiene un error de sintaxis, autenticación o recurso inexistente.

400 Bad Request
Sintaxis inválida o JSON mal formado enviado al servidor.

401 Unauthorized
Falta autenticación o credenciales de sesión válidas.

403 Forbidden
Autenticado pero sin permisos para acceder al recurso.

404 Not Found
La URL o recurso solicitado no existe en el servidor.

🔒 **DIFERENCIA ENTRE 401 Y 403**
401: No sé quién sos (iniciá sesión) · **403:** Sé quién sos, pero no tenés el rol o permiso para entrar acá.

BLOQUE 4 | ERRORES DEL SERVIDOR

Códigos 5×x: Errores del Servidor

El cliente envió una petición correcta pero el servidor falló al intentar procesarla.

500 Internal Server Error

Excepción no controlada o fallo genérico en el backend.

502 Bad Gateway

Respuesta inválida recibida desde un servidor intermedio.

503 Service Unavailable

Servidor colapsado por tráfico o en mantenimiento.

504 Gateway Timeout

Tiempo de espera agotado en la comunicación de proxy.

DIAGNÓSTICO BACKEND

Un error 500 no se resuelve modificando el cliente; requiere revisar los logs de código del servidor backend.

BLOQUE 5 | SEGURIDAD WEB

HTTP vs HTTPS: Por Qué es Indispensable Cifrar

HTTPS es la versión segura de HTTP respaldada por la capa de cifrado TLS/SSL.

HTTP (Puerto 80)

Transmisión en **texto plano** legibles por cualquier nodo intermedio de red.

Vulnerable a ataques y captura de contraseñas.

HTTPS (Puerto 443)

Cifrado de extremo a extremo mediante algoritmos criptográficos TLS.

Garantiza confidencialidad, autenticidad e integridad de datos.

PELIGRO EN REDES WI-FI PÚBLICAS

En un Wi-Fi abierto, cualquier usuario con un analizador de paquetes (Wireshark) puede leer tus contraseñas si navegás sobre HTTP no cifrado.

BLOQUE 5 | TLS Y CERTIFICADOS

TLS/SSL y Certificados Digitales

Los certificados digitales emitidos por Autoridades de Certificación (CA) validan la identidad.

🔑Proceso de Cifrado Híbrido

Cifrado Asimétrico: Clave pública y privada para negociar la sesión inicial de forma segura.

Cifrado Simétrico: Utiliza una clave única de sesión ultrarrápida para cifrar el tráfico de datos.



CANDADO VERDE DEL NAVEGADOR

El candado confirma que el certificado fue emitido por una CA válida y que las claves criptográficas están activas.

BLOQUE 5 | GESTIÓN DE ESTADO

Cookies y Gestión de Estado

El protocolo HTTP es "stateless" (sin estado); las cookies permiten recordar sesiones de usuario.

Guardado: Set-Cookie

El servidor envía la cabecera Set-Cookie: session_id=abc123 en la respuesta para almacenar un identificador.

Envío: Cookie Header

El navegador reenvía automáticamente la cabecera Cookie: session_id=abc123 en cada petición subsiguiente.

MANTENER LA SESIÓN ACTIVA

Gracias a las cookies, podés navegar entre 50 productos de un e-commerce sin tener que volver a ingresar tu usuario y contraseña en cada clic.

BLOQUE 5 | SEGURIDAD EN COOKIES

Atributos de Seguridad de las Cookies

Flags de configuración fundamentales para proteger los tokens de sesión frente a ataques.

</>HttpOnly

Impide que scripts de JavaScript (document.cookie) accedan a la cookie. Mitiga ataques XSS.

Secure

Instruye al navegador a transmitir la cookie únicamente a través de conexiones cifradas HTTPS.

SameSite

Restringe el envío de la cookie en peticiones entre distintos sitios. Mitiga ataques CSRF.

CONFIGURACIÓN RECOMENDADA DE PRODUCCIÓN

Set-Cookie: session_id=xyz; Secure; HttpOnly; SameSite=Strict es el estándar de oro para proteger cookies bancarias y de autenticación.

Cliente-Servidor

Flujo indivisible de petición y respuesta en 5 etapas.

Protocolo HTTP

Métodos semánticos, cabeceras, payload y códigos de estado.

HTTPS y Sesiones

Cifrado TLS/SSL, certificados CA y cookies seguras.

Image Sources



https://media.istockphoto.com/id/2149328154/vector/lan-network-3d-diagram.jpg?s=612x612&w=0&k=20&c=0sWs7plVl8iAwxR1eNJVwWEROHpmzkNcM3BuGg4nQOg=

Source: www.istockphoto.com



https://media.geeksforgeeks.org/wp-content/uploads/20240111152750/HTTPS-gif.gif

Source: www.geeksforgeeks.org